

Best Time to Buy & Sell Stock

A Step-by-Step Visual Walkthrough

Brute Force → Optimal $O(n)$ Solution

Presented by:

Atul Arora | **Vikrant Garje**



Problem Statement

You are given an array `prices[]` where `prices[i]` is the price of a stock on the *i*-th day.

Choose a single day to BUY and a different future day to SELL.

Return the maximum profit. If no profit is possible, return 0.

Example Input:

```
prices = [7, 1, 5, 3, 6, 4]
```

Expected Output:

```
5 (Buy at day 2 @ 1, Sell at day 5 @ 6)
```

Must buy BEFORE you sell | Only ONE transaction allowed



Brute Force Approach

💡 Idea

Try EVERY pair (i, j) where $i < j$
Calculate profit = prices[j] - prices[i]
Track the maximum profit seen

Complexity

Time: $O(n^2)$ | Space: $O(1)$

// C++ - Brute Force

```
int maxProfit(int arr[], int n) {  
    int maxP = 0;  
    for (int i = 0; i < n-1; i++) {  
        for (int j = i+1; j < n; j++) {  
            int profit = arr[j] - arr[i];  
            maxP = max(maxP, profit);  
        }  
    }  
    return maxP;  
}
```

❖❖ Brute Force – Trace | arr = [7, 1, 5, 3, 6, 4]

Array:

7	1	5	3	6	4
[0]	[1]	[2]	[3]	[4]	[5]

i	j	arr[i] → arr[j]	Profit = arr[j] - arr[i]	maxProfit	Note
0	1	7→1	-6	0	Loss, skip
0	2	7→5	-2	0	Loss, skip
0	3	7→3	-4	0	Loss, skip
0	4	7→6	-1	0	Loss, skip
0	5	7→4	-3	0	Loss, skip
1	2	1→5	+4	4	New max!
1	3	1→3	+2	4	No change
1	4	1→6	+5	5	New max!

(Continued on next slide...)

❖❖ Brute Force – Trace (continued) | arr = [7, 1, 5, 3, 6, 4]

i	j	arr[i] → arr[j]	Profit = arr[j] - arr[i]	maxProfit	Note
1	5	1→4	+3	5	No change
2	3	5→3	-2	5	Loss, skip
2	4	5→6	+1	5	No change
2	5	5→4	-1	5	Loss, skip
3	4	3→6	+3	5	No change
3	5	3→4	+1	5	No change
4	5	6→4	-2	5	Loss, skip

❖❖ Brute Force Result

Maximum Profit = 5 (Buy at index 1 @ price 1, Sell at index 4 @ price 6)

Total pairs checked: 15 = 15 comparisons ($O(n^2)$)

Optimal Approach – Single Pass $O(n)$

◆◆ Key Insight

Track the minimum price seen so far.
At each day, calculate profit if we
sold TODAY vs the lowest buy price.
Only ONE forward pass needed!

Time: $O(n)$ | Space: $O(1)$

// C++ – Optimal $O(n)$

```
int maxProfit(int arr[], int n) {  
    if (n < 2) return 0;  
    int minPrice = arr[0];  
    int maxProfitSoFar = 0;  
    for (int i = 0; i < n; i++) {  
        minPrice = min(minPrice, arr[i]);  
        maxProfitSoFar = max(maxProfitSoFar,  
                             arr[i] - minPrice);  
    }  
    return maxProfitSoFar;  
}
```

Optimal Trace – arr = [7, 1, 5, 3, 6, 4] (Single Pass)

Array:

7	1	5	3	6	4
[0]	[1]	[2]	[3]	[4]	[5]

↑ BUY

↑ SELL

i	arr[i]	minPrice	Profit=arr[i]-min	maxProfit	Updated?	Remark
0	7	7	7-7=0	0	–	Init minPrice=7
1	1	1	1-1=0	0	min	New min! minPrice=1
2	5	1	5-1=4	4	profit	maxProfit updated!
3	3	1	3-1=2	4	–	Profit < max, skip
4	6	1	6-1=5	5	profit	Best profit: 5 💎
5	4	1	4-1=3	5	–	Profit < max, skip

💎 Max Profit = 5 | Only 6 iterations! (vs 15 in brute force)



Brute Force vs Optimal – Comparison



Brute Force

- Two nested for loops
- Time: $O(n^2)$
- Space: $O(1)$
- $n=6$: 15 iterations
- $n=1000$: ~500,000 iters
- Slow for large inputs

VS

Optimal

- Single for loop
- Time: $O(n)$
- Space: $O(1)$
- $n=6$: 6 iterations
- $n=1000$: 1,000 iters
- Fast for large inputs

◆◆ For $n=1,000,000$ stocks – Brute Force: ~500 Billion ops | Optimal: 1 Million ops

Key Takeaways

01

Brute Force is intuitive

Try all pairs – $O(n^2)$ time – good for understanding the problem

02

Track the minimum

Key insight: always know the cheapest price seen so far

03

Single pass wins

Optimal scans once left-to-right, updating min and maxProfit – $O(n)$

04

Same answer, less work

Both return 5 for $[7,1,5,3,6,4]$ – optimal does it in 6 steps vs 15